A Practical Guide to Monte Carlo tree search

TOPIC -- MONTE CARLO TREE SEARCH

-- 01 --

In 2006, inspired by its predecessors, Rémi Coulom described the application of the Monte Carlo method to game-tree search and coined the name Monte Carlo tree search, L. Kocsis and Cs. Szepesvári developed the UCT (Upper Confidence bounds applied to Trees) algorithm, and S. Gelly et al. implemented UCT in their program MoGo. In 2008, MoGo achieved dan (master) level in 9×9 Go, and the Fuego program began to win against strong amateur players in 9×9 Go. In January 2012, the Zen program won 3:1 in a Go match on a 19×19 board with an amateur 2 dan player. Google Deepmind developed the program AlphaGo, which in October 2015 became the first Computer Go program to beat a professional human Go player without handicaps on a full-sized 19x19 board. In March 2016, AlphaGo was awarded an honorary 9-dan (master) level in 19×19 Go for defeating Lee Sedol in a five-game match with a final score of four games to one. AlphaGo represents a significant improvement over previous Go programs as well as a milestone in machine learning as it uses Monte Carlo tree search with artificial neural networks (a deep learning method) for policy (move selection) and value, giving it efficiency far surpassing previous programs. The MCTS algorithm has also been used in programs that play other board games (for example Hex, Havannah, Game of the Amazons, and Arimaa), real-time video games (for instance Ms. Pac-Man and Fable Legends), and nondeterministic games (such as skat, poker, Magic: The Gathering, or Settlers of Catan).

-- 02 --

When using RAVE, the selection step selects the node, for which the modified UCB1 formula (1 – β (n i , n ~ i)) w i n i + β (n i , n ~ i) w ~ i n ~ i + c ln ■ t n i {\displaystyle (1-\beta (n_{i},{\tilde {n}}_{i})){\frac {w_{i}}{n_{i}}}+\beta (n_{i},{\tilde {n}}_{i}){\frac {{\tilde {w}}_{i}}{{\tilde {n}}_{i}}}+c{\sqrt {\frac {\ln t}{n_{i}}}}} has the highest value. In this formula, w ~ i {\displaystyle {\tilde {w}}_{i}} and n ~ i {\displaystyle {\tilde {n}}_{i}} stand for the number of won playouts containing move i and the number of all playouts containing move i, and the β (n i , n ~ i) {\displaystyle \beta (n_{i},{\tilde {n}}_{i})} function should be close to one and to zero for relatively small and relatively big ni and n ~ i {\displaystyle {\tilde {n}}_{i}} , respectively. One of many formulas for β (n i , n ~ i) {\displaystyle \beta (n_{i},{\tilde {n}}_{i})} , proposed by D. Silver, says that in balanced positions one

can take β (n i , n ~ i) = n ~ i n i + n ~ i + 4 b 2 n i n ~ i {\displaystyle \beta (n_{i},{\tilde {n}}_{i})={\frac {{\tilde {n}}_{i}}{n_{i}+{\tilde {n}}_{i}+4b^{2}n_{i}{\tilde {n}}_{i}}}} , where b is an empirically chosen constant. Heuristics used in Monte Carlo tree search often require many parameters. There are automated methods to tune the parameters to maximize the win rate. Monte Carlo tree search can be concurrently executed by many threads or processes. There are several fundamentally different methods of its parallel execution:

-- 03 --

Selection: Start from root R and select successive child nodes until a leaf node L is reached. The root is the current game state and a leaf is any node that has a potential child from which no simulation (playout) has yet been initiated. The section below says more about a way of biasing choice of child nodes that lets the game tree expand towards the most promising moves, which is the essence of Monte Carlo tree search. Expansion: Unless L ends the game decisively (e.g. win/loss/draw) for either player, create one (or more) child nodes and choose node C from one of them. Child nodes are any valid moves from the game position defined by L. Simulation: Complete one random playout from node C. This step is sometimes also called playout or rollout. A playout may be as simple as choosing uniform random moves until the game is decided (for example in chess, the game is won, lost, or drawn). Backpropagation: Use the result of the playout to update information in the nodes on the path from C to R.

-- 04 --

Exploration and exploitation The main difficulty in selecting child nodes is maintaining some balance between the exploitation of deep variants after moves with high average win rate and the exploration of moves with few simulations. The first formula for balancing exploitation and exploration in games, called UCT (Upper Confidence Bound 1 applied to trees), was introduced by Levente Kocsis and Csaba Szepesvári. UCT is based on the UCB1 formula derived by Auer, Cesa-Bianchi, and Fischer and the probably convergent AMS (Adaptive Multi-stage Sampling) algorithm first applied to multi-stage decision-making models (specifically, Markov Decision Processes) by Chang, Fu, Hu, and Marcus. Kocsis and Szepesvári recommend to choose in each node of the game tree the move for which the expression w i n i + c ln ■ N i n i {\displaystyle {\frac {w_{i}}{n_{i}}}+c{\sqrt {\frac {\ln N_{i}}{n_{i}}}}} has the highest value. In this formula:

-- 05 --

This graph shows the steps involved in one decision, with each node showing the ratio of wins to total playouts from that point in the game tree for the player that the node represents. In the Selection diagram, black is about to move. The root node shows there are 11 wins out of 21 playouts for white from this position so far. It complements the total of 10/21 black wins shown along the three black nodes under it, each of which represents a possible black move. Note that this graph does not follow the UCT algorithm described below. If white loses the simulation, all nodes along the selection incremented their simulation count (the denominator), but among them only the black nodes were credited with wins (the numerator). If instead white wins, all nodes along the selection would still increment their simulation count, but among them only the white nodes would be credited with wins. In games where draws are possible, a draw causes the numerator for both black and white to be incremented by 0.5 and the denominator by 1. This ensures that during selection, each player's choices expand towards the most promising moves for that player, which mirrors the goal of each player to maximize the value of their move. Rounds of search are repeated as long as the time allotted to a move remains. Then the move with the most simulations made (i.e. the highest denominator) is chosen as the final answer.